

Recap: Chat, Cowork, Code and the models behind them

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 4 of 12 · English notes

Session 4 (10 August 2026) opened with a recap of the three client surfaces already in play: **Claude Chat** (claude.ai), **Claude Cowork** (desktop), and **Claude Code** (terminal / IDE agent). The trainer, Vijay, then used that map to continue end-to-end generation of a task-management app — still without writing application code by hand.

CLASSROOM UI VERSUS CURRENT DOCS

Labels, model names (Haiku / Sonnet / Opus families), and Claude Code slash menus in the August 2026 recording will drift. For the exam and for delivery work, treat [docs.claude.com](#) and **Claude Code docs** as the source of truth.

The three products in one sentence each

The recap is not three unrelated apps. They share the same model families and many of the same memory patterns (CLAUDE . md, skills). They differ in *where the work happens* and how much the client is allowed to act on your machine.

Product	What the trainer emphasised	Typical use in this cohort
Claude Chat (claude.ai)	Chat-style Q&A; pick Haiku, Sonnet, or Opus (or the live equivalent) and get a response.	Exploration, drafting, questions that do not need a repo.
Claude Cowork	Desktop client: folders, repetitive tasks, templates, and (in this session) Design / wireframes.	Organising working folders; generating artefacts that later feed Code.
Claude Code	Agentic coding: plan, tools, sandbox, production-shaped apps from prompts, skills, and memory files.	Full-stack generation, OpenAPI-first backends, MCP-backed front ends.

Cowork is not “Chat with a nicer window.” It is closer to an agent that can work on local folders. Claude Code is the same idea aimed at a software repo: an agent loop that can read, write, run commands, and verify. Chat remains the lightest surface: you ask, the model answers, and it does not own your project tree.

Models: Haiku, Sonnet, Opus — and why the names are not the exam

In the recap the trainer named the three families as HiQ/Haiku, Sonnet, and Opus. The teaching point is the *ladder*, not a frozen SKU list: a cheaper/faster model for routine edits, a mid-tier default for most coding, and a heavier model when planning or architecture needs more reasoning. In Claude Code you switch with `/model` (covered in Module 05). Org availability still wins: you only see models your workspace is entitled to.

- Same families can sit behind Chat, Cowork, and Code; the *client* changes tool access and persistence, not the idea of “which brain.”

- A live session may show a default such as “Sonnet 5”; do not memorise that string. Memorise: pick the smallest model that still does the job, and check the live model page before the exam.
- Token cost follows model choice *and* how much you attach (whole folders versus a single OpenAPI YAML).

Memory that is universal: CLAUDE.md versus skills

The trainer treated CLAUDE .md and skills as *universal* patterns: they apply across Cowork, Code, and (where the product supports project memory) Chat-adjacent workflows. A learner asked the difference in this session; the answer is the one to keep for the exam.

Mechanism	When it is applied	How you invoke it
CLAUDE.md (memory)	Appended by default on requests the client makes to the model.	You do not slash it. Put it at repo root; optional extra files in frontend/backend folders.
Skill (SKILL.md in a named skill folder)	Only when the skill is in play.	Slash command (e.g. /business-analyst, /architect), unless auto-invocation is left on.

CLAUDE .md is standing system instruction: production-ready code, language best practices, single responsibility, “for all diagrams use Mermaid,” and so on. Nested files let backend and frontend carry different house rules. Skills encode a *role* (business analyst, architect, OpenAPI designer, frontend engineer). The file name the trainer used for the skill body is SKILL .md (capitalised) inside a skill folder; once defined, Claude Code exposes a matching slash command.

AUTO-INVOCATION OF SKILLS

By default Claude may pick a skill when your wording matches it (SRS, “business analyst,” and so on). To force slash-only use, set the skill property that disables model invocation (the trainer’s phrase: *disable model invocation* true). When an agentic loop is running, skills can also be pulled in without you typing the slash — that is intentional, not a bug.

SDLC roles as skills, not as extra models

The live demo continued the task-management app as if a real SDLC were staffed by Claude roles: sales/stakeholders produce a BRD/SRS via a business-analyst skill; architects produce HLD, component and sequence diagrams, OpenAPI; developers consume those artefacts. You still review and publish (PDF, Confluence). Claude drafts; humans own the document.

1. Requirements skill: in/out of scope, login vs register, dashboard, CRUD on tasks.
2. Architect skill: think like a principal solution architect; HLD, deployment, data flow, NFRs (performance, security, availability).
3. Prefer Mermaid (or similar text diagrams) over JPEG/SVG generation: fewer tokens, editable as code, GitHub and Confluence render it.
4. Same slash commands work in the VS Code Claude UI and in the terminal c laude session; Mermaid preview in the IDE needs an extension, GitHub does not.

The five-phase agent loop (Chat, Cowork, and Code)

Whenever you give a substantial task to Cowork or Code, the trainer described a five-step pattern he also called a chain-of-thought style execution: **clarify → plan → execute → verify → deliver**. Example: “organise my Downloads folder by file type.” Ambiguity produces questions (keep original names? duplicates?). Then a plan (scan, categorise images/docs/archives), then file moves, then a check that nothing was lost, then a summary.

- Verification (clean-code review, full test regen) is expensive. Many teams put “do not auto-verify generated code” in memory and test locally instead.
- Sequence can be constrained in CLAUDE.md (“finish controllers before services”).
- Always read destructive commands. Consent is yours; Anthropic will not own a delete you approved.
- Prompt engineering named “chain of thought” is deferred to later modules; here it is the *behaviour* of the agent loop, not an API flag.

What to carry into the next chapters

Module 04 then leaves the GUI recap and spends the rest of the recording on design-first APIs and MCP. If you only remember three facts from this chapter: (1) Chat / Cowork / Code are one product family with different agency; (2) CLAUDE.md is always-on memory, skills are role packets; (3) the agent loop is clarify–plan–execute–verify–deliver, and verification is a token tax you can turn down.

Official documentation

- **Claude Docs home**

<https://docs.claude.com/en/home>

Start here when a classroom label no longer matches the product.

- **Models**

<https://docs.claude.com/en/docs/about-claude/models>

Live Haiku / Sonnet / Opus (and successor) names and context windows.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Terminal and IDE agent used throughout this module.

- **Cowork / Claude desktop support**

<https://support.claude.com/>

Desktop client, Design, and account access issues.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.